

Coding Standards and Best Practices Guide - .NET Standard Track

Introduction

Coding standards are rules and guidelines that ensure code is consistent, readable, and high-quality. They improve a codebase by enhancing readability and maintainability, simplifying collaboration among developers, and reducing bugs. Adhering to these practices makes code safer, more secure, and easier to scale.

What are coding standards and best practices?

- **A set of guidelines:** Coding standards are a framework of rules for writing code, ensuring consistency across a project and development team.
- **A roadmap for quality:** They act as blueprints to create safe, secure, reliable, and maintainable software.
- **Vary by context:** Standards can be specific to a programming language, industry, or even an individual project.

Key benefits of coding standards

- **Improved readability and maintainability:** Consistent formatting, naming conventions, and structure make code easier to read, understand, and update over time.
- **Enhanced collaboration:** Standards create a uniform style and set of assumptions, allowing team members to work together more effectively.
- **Reduced errors:** Following best practices helps prevent bugs and makes it easier to find and fix problems.
- **Faster onboarding:** New team members can get up to speed more quickly if the codebase follows a consistent and understandable standard.
- **Increased efficiency:** They help programmers write cleaner, more efficient code and reduce technical debt.

Module: Agile Methodology

Agile methodology best practices

- **Build around motivated individuals:** Trust and empower self-organizing, cross-functional teams.
- **Prioritize communication:** Encourage face-to-face conversation, as it's the most efficient way to convey information.
- **Iterative and incremental delivery:** Focus on delivering working software frequently, in short cycles, to get continuous feedback.
- **Embrace change:** Be open to changing requirements at any stage of the project.
- **Continuous improvement:** Reflect on and adapt processes regularly to improve effectiveness.
- **Automate testing:** Implement automated tests to catch bugs early and ensure code quality with every change.

Agile coding standards and practices

- **Write clean code:** Ensure code is readable, understandable, and well-documented.
- **Enforce a single coding standard:** Agree on a common set of coding standards for the entire team to maintain consistency.
- **Use test-driven development (TDD):** Write automated tests *before* writing the code they are meant to test.
- **Refactor continuously:** Regularly improve the design and clarity of existing code to manage technical debt.
- **Practice pair programming:** Have two developers work together on one machine to improve code quality and knowledge sharing.
- **Integrate continuously:** Use continuous integration to automatically build and test code frequently.
- **Use simple design:** Aim for the simplest design that can achieve the goal, minimizing unnecessary complexity.

- **Regularly review code:** Implement a process for other team members to review code before it's merged into the main branch.

Module: Microsoft SQL Server

Microsoft SQL Server best practices and coding standards encompass various aspects, from database design and server configuration to query optimization and security. Adhering to these guidelines helps ensure performance, maintainability, and data integrity.

Database Design and Development:

- **Normalization:**

Design databases to at least Third Normal Form (3NF) for OLTP systems to reduce data redundancy and improve data integrity.

- **Naming Conventions:**

Employ consistent and descriptive naming conventions for tables, columns, stored procedures, and other database objects.

- **Data Types:**

Use appropriate data types for columns to optimize storage and performance (e.g., `DATETIME2` for date and time, `NVARCHAR` for Unicode strings).

- **Indexes:**

Create appropriate indexes (clustered and non-clustered) to improve query performance, avoiding duplicate or unnecessary indexes.

- **Foreign Keys:**

Utilize foreign keys to enforce referential integrity and maintain relationships between tables.

- **Stored Procedures:**

Use stored procedures for complex or frequently executed logic to reduce network round trips and improve performance.

- **Transactions:**

Implement transactions following ACID principles (Atomicity, Consistency, Isolation, Durability) to ensure data integrity, using appropriate isolation levels.

- **Error Handling:**

Include robust error handling mechanisms within stored procedures and other database code.

Query Optimization and Performance:

- **Avoid `SELECT *`:**
Explicitly list required columns in `SELECT` statements to reduce network traffic and improve performance.
- **Minimize Temporary Tables:**
Use Common Table Expressions (CTEs) or derived tables instead of temporary tables when possible.
- **Efficient Joins:**
Use explicit `JOIN` syntax and fully qualify table and column names in joins.
- **Avoid Cursors:**
Use set-based operations (e.g., `INSERT INTO...SELECT`) instead of cursors or application-level loops for data manipulation.
- **`SET NOCOUNT ON`:**
Include `SET NOCOUNT ON` at the beginning of stored procedures to prevent sending row count information back to the client, reducing network traffic.
- **Index Usage:**
Ensure queries effectively utilize existing indexes.
- **Query Optimization:**
Use SQL Server tools (e.g., Execution Plans) to analyze and optimize query performance.

Server Configuration and Security:

- **Dedicated Server:**
Use a dedicated server for SQL Server when possible, especially in production environments.
- **Storage Configuration:**
Separate data and log files onto different disks or filegroups with appropriate RAID levels (e.g., RAID 10 for data and logs).
- **TempDB Configuration:**
Configure `tempdb` appropriately, including placing it on fast storage and sizing it correctly.
- **Security:**

Implement security best practices such as least privilege, strong authentication, Transparent Data Encryption (TDE) for data at rest, and regular auditing.

- **Backups:**

Establish a comprehensive backup and recovery strategy.

Code Readability and Maintainability:

- **Consistent Formatting:**

Use consistent formatting, indentation, and capitalization for SQL keywords.

- **Comments:**

Include comments in SQL code to explain complex logic or non-obvious design choices.

- **Modularity:**

Break down complex logic into smaller, manageable units (e.g., separate stored procedures for distinct tasks).

Module: C#

C# best practices and coding standards aim to produce clean, readable, maintainable, and efficient code. Adhering to these guidelines enhances collaboration and reduces errors.

General Principles:

- **Readability and Clarity:** Prioritize code that is easy to understand.
- **Consistency:** Maintain a consistent style throughout the codebase.
- **Simplicity:** Favor simpler solutions over complex ones.
- **DRY (Don't Repeat Yourself):** Avoid code duplication.
- **SOLID Principles:** Adhere to Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion principles.

Naming Conventions:

- **PascalCase:** Use for classes, methods, properties, enums, and public fields.
- **camelCase:** Use for local variables and method parameters.
- **Meaningful Names:** Choose descriptive names that clearly convey purpose, avoiding abbreviations unless widely recognized.

- **Private Fields:** Start with an underscore (`_`) and use camelCase (e.g., `_myPrivateField`).

Code Structure and Formatting:

- **Indentation:** Use consistent indentation (e.g., 4 spaces or tabs, depending on team preference).
- **Braces:** Always use braces for `if`, `else`, `for`, `while` statements, even for single-line blocks.
- **Whitespace:** Use whitespace to improve readability, such as around operators and between logical blocks of code.
- **Line Length:** Strive to keep lines within a reasonable length (e.g., 120 characters).
- **File-scoped namespaces:** Utilize file-scoped namespaces to reduce indentation.
- **Region Directives:** Use `#region` and `#endregion` sparingly and thoughtfully to organize code, not hide complexity.

Specific Practices:

- **Method Design:**
 - Methods should ideally perform a single, well-defined task.
 - Keep methods concise; refactor long methods into smaller, more focused ones.
- **Exception Handling:**
 - Use `try-catch` blocks to handle expected exceptions gracefully.
 - Catch specific exception types rather than general `System.Exception` where possible.
- **Modern Language Features:**

Utilize modern C# features like `async/await` for asynchronous operations and LINQ for collection manipulation.
- **Dependency Injection:**

Favor dependency injection over hardcoding dependencies to improve testability and modularity.
- **Unit Testing:**

Write comprehensive unit tests to ensure code correctness and facilitate refactoring.

- **Comments:**

Use comments to explain why a piece of code is written in a particular way, not what it does (which should be clear from the code itself). Use XML documentation comments for public APIs.

- **Performance and Security:**

- Optimize code for performance where necessary, using appropriate data structures and algorithms.
- Implement secure coding practices, such as parameterized queries to prevent SQL injection and input validation.

Module: ASP.NET Core MVC

ASP.NET Core MVC best practices and coding standards encompass various aspects, from application architecture to performance and security.

Architectural and Design Principles:

- **Separation of Concerns (SoC):**

Adhere strictly to the MVC pattern. Keep controllers lean, focusing on request handling and view model preparation. Delegate business logic to services and data access to repositories.

- **Dependency Injection:**

Utilize ASP.NET Core's built-in dependency injection to manage service lifetimes and promote testability and modularity.

- **View Models:**

Create specific View Models for each view to encapsulate the data required for that view, avoiding direct exposure of domain models.

- **Asynchronous Programming:**

Employ `async` and `await` for I/O-bound operations (e.g., database calls, external API requests) to improve scalability and responsiveness.

- **Error Handling:**

Implement robust error handling, including global exception handling and custom error pages, to provide a consistent user experience and prevent sensitive information disclosure.

Performance Optimization:

- **Caching:**

Implement caching strategies (e.g., response caching, in-memory caching) for frequently accessed or computationally expensive data to reduce load and improve response times.

- **Bundling and Minification:**

Minify client-side assets (CSS, JavaScript) and bundle them to reduce file sizes and the number of HTTP requests.

- **Efficient Data Access:**

Optimize database queries, use asynchronous methods for data access, and retrieve only necessary data to minimize overhead. Consider using `AsNoTracking()` for read-only queries with Entity Framework.

- **Compression:**

Enable response compression (e.g., GZip) to reduce the size of data transferred over the network.

Security:

- **Input Validation:**

Validate all user input to prevent common vulnerabilities like XSS and SQL injection.

- **Authentication and Authorization:**

Implement secure authentication and authorization mechanisms, enforcing strong password policies and considering multi-factor authentication.

- **Secure Data Storage:**

Never store sensitive data (e.g., passwords) in plain text; use hashing and encryption.

- **Keep Software Up-to-Date:**

Regularly update ASP.NET Core, third-party libraries, and the operating system to patch security vulnerabilities.

Coding Standards and Maintainability:

- **Consistent Naming Conventions:**

Follow C# naming conventions (e.g., PascalCase for classes and methods, camelCase for local variables).

- **Code Readability:**

Write clear, concise, and well-commented code.

- **Refactoring:**

Regularly refactor auto-generated or complex code to improve maintainability.

- **Automated Testing:**

Implement unit tests, integration tests, and end-to-end tests to ensure code quality and prevent regressions.

- **Logging:**

Implement comprehensive logging to monitor application behavior and aid in debugging.

Module: ASP.NET Core Web API

ASP.NET Core Web API best practices and coding standards encompass various aspects, including performance, security, maintainability, and API design.

Performance Optimization:

- **Asynchronous Programming (async/await):**

Utilize asynchronous operations extensively to prevent blocking threads and improve scalability, especially for I/O-bound tasks.

- **Caching:**

Implement caching strategies (in-memory, distributed) for frequently accessed data to reduce database calls and improve response times.

- **Pagination and Filtering:**

For large datasets, implement pagination, filtering, and sorting options to avoid returning excessive data in a single response.

- **Minimize Large Object Allocations:**

Be mindful of object creation and disposal to reduce garbage collection overhead.

- **Optimize Data Access:**

Use efficient data access patterns with Entity Framework Core, such as `AsNoTracking()` for read-only queries.

Security:

- **Authentication and Authorization:** Implement robust authentication (e.g., JWT, OAuth) and authorization mechanisms to control access to API resources.
- **Input Validation and Sanitization:** Validate and sanitize all user input to prevent common vulnerabilities like SQL injection and XSS.
- **Secure Cookies and Sessions:** Use secure and HTTP-only cookies, and manage sessions securely.
- **HTTPS:** Enforce HTTPS for all API communication to protect data in transit.
- **Error Handling and Logging:** Implement centralized error handling and log errors securely without exposing sensitive information.

Maintainability and Code Quality:

- **Clean Architecture/Separation of Concerns:**
Design APIs with a clear separation of concerns (e.g., controllers, services, repositories) to improve modularity and testability.
- **Dependency Injection:**
Utilize dependency injection for managing dependencies and promoting loose coupling.
- **Meaningful Naming Conventions:**
Use clear and consistent naming conventions for controllers, actions, models, and variables.
- **Code Comments and Documentation:**
Provide clear comments for complex logic and generate API documentation (e.g., using Swagger/OpenAPI).
- **Unit and Integration Testing:**
Write comprehensive unit and integration tests to ensure code correctness and prevent regressions.

API Design:

- **RESTful Principles:**
Adhere to RESTful principles, using appropriate HTTP verbs (GET, POST, PUT, DELETE) and resource-based URLs (nouns).
- **Consistent Response Formats:**

Use a consistent format for API responses, typically JSON, and provide informative status codes.

- **Version Control:**

Implement API versioning to manage changes and ensure backward compatibility.

- **Error Handling:**

Provide clear and consistent error responses with appropriate HTTP status codes and error details.

- **Documentation:**

Maintain up-to-date and comprehensive API documentation for consumers.

General Coding Standards:

- **Follow C# Coding Conventions:**

Adhere to standard C# coding conventions (e.g., PascalCase for classes and methods, camelCase for local variables).

- **Avoid Synchronous I/O:**

Do not perform synchronous read/write operations on `HttpRequest/HttpResponse` bodies.

- **Minimize Exceptions:**

Use exceptions for exceptional circumstances, not for routine control flow.

- **Keep Dependencies Updated:**

Regularly update NuGet packages and framework versions to benefit from bug fixes and performance improvements.

Module: Microservices Architecture

Developing .NET microservices involves adhering to best practices and coding standards to ensure maintainability, scalability, and resilience.

Best Practices for .NET Microservices:

- **Domain-Driven Design (DDD):**

Model services around business domains and bounded contexts to create clear service boundaries and reduce coupling.

- **Single Responsibility Principle:**

Each microservice should have a single, well-defined responsibility, preventing bloated services and simplifying maintenance.

- **Data Independence:**

Each microservice should own its data store, promoting autonomy and isolating data-related changes.

- **Well-Defined APIs:**

Design robust, versioned, and well-documented APIs (RESTful or gRPC) for inter-service communication. Utilize OpenAPI (Swagger) for documentation.

- **Asynchronous Communication:**

Favor asynchronous communication patterns (e.g., message queues like RabbitMQ or Kafka) for loose coupling and improved resilience.

- **Observability:**

Implement centralized logging, distributed tracing (e.g., OpenTelemetry), and metrics collection to gain insights into service behavior and quickly identify issues.

- **Fault Isolation and Resilience:**

Design services to be resilient to failures using patterns like circuit breakers, retries, and bulkheads.

- **Containerization:**

Package microservices in containers (e.g., Docker) for consistent deployment environments and simplified management.

- **CI/CD:**

Automate testing and deployment through Continuous Integration and Continuous Deployment pipelines.

- **Security:**

Implement robust security measures, including authentication, authorization (e.g., JWT tokens), and secure communication protocols (e.g., mTLS).

- **API Gateway:**

Utilize an API Gateway (e.g., Ocelot) to handle cross-cutting concerns like authentication, routing, and rate limiting.

Coding Standards for .NET Microservices:

- **Consistent Naming Conventions:**

Adhere to established .NET naming conventions for classes, methods, variables, and properties.

- **Code Readability:**

Write clear, concise, and self-documenting code. Use meaningful variable names and avoid excessive comments where code clarity suffices.

- **Error Handling:**

Implement robust error handling mechanisms, including structured logging of exceptions and graceful degradation.

- **Dependency Injection:**

Utilize dependency injection extensively for managing dependencies and promoting testability.

- **Asynchronous Programming:**

Employ `async` and `await` for I/O-bound operations to improve responsiveness and scalability.

- **Configuration Management:**

Externalize configuration settings to environment variables or dedicated configuration services (e.g., Azure App Configuration) for easy management.

- **Unit and Integration Testing:**

Write comprehensive unit and integration tests to ensure code quality and prevent regressions.

- **Performance Optimization:**

Be mindful of performance-critical areas, avoid unnecessary allocations, and optimize data access.

- **Code Analysis Tools:**

Utilize static code analysis tools (e.g., Roslyn analyzers, SonarQube) to enforce coding standards and identify potential issues.

- **Minimize Technical Debt:**

Regularly refactor and address technical debt to maintain a healthy codebase.

Module: Entity Framework Core

Entity Framework Core (EF Core) best practices and coding standards focus on performance, maintainability, and correct data handling.

Performance Optimization:

- **Asynchronous Operations:**

Utilize `async/await` with EF Core's asynchronous methods (e.g., `FirstOrDefaultAsync`, `ToListAsync`) to prevent blocking the main thread during I/O operations, improving application responsiveness.

- `AsNoTracking()` for Read-Only Queries:

When entities are not intended for updates, use `AsNoTracking()` to prevent EF Core from tracking changes, significantly improving query performance and reducing memory consumption.

- Projection with `Select()`:

Instead of fetching entire entities, use `Select()` to project only the necessary columns into a DTO (Data Transfer Object) or anonymous type, minimizing data transfer and memory usage.

- Eager Loading with `Include()` and `ThenInclude()`:

For related data, use `Include()` and `ThenInclude()` to eagerly load related entities in a single query, avoiding the N+1 query problem and reducing database roundtrips.

- Split Queries with `AsSplitQuery()`:

For queries involving multiple `Include` statements on different root entities, consider `AsSplitQuery()` to break down the query into multiple SQL queries, which can improve performance and reduce the risk of Cartesian explosions.

- **Efficient Pagination:**

Implement server-side pagination using `Skip()` and `Take()` to retrieve only the required subset of data, preventing the loading of large datasets into memory.

- **Batch Saves:**

Minimize database roundtrips by performing multiple updates or inserts within a single `SaveChanges()` or `SaveChangesAsync()` call.

Maintainability and Correctness:

- **Clear Entity Naming:**

Use clear and descriptive names for entity classes and properties that accurately reflect their purpose.

- **Data Annotations vs. Fluent API:**

Choose the appropriate configuration method for your model. Data Annotations are suitable for simple configurations, while Fluent API provides more flexibility for complex mappings and advanced configurations.

- **Migrations:**

Utilize EF Core migrations for managing database schema changes, ensuring consistency between your code and the database.

- **Secure Connection Strings:**

Protect sensitive connection string information by storing it securely, for example, in environment variables or a secrets manager, and not directly in source code.

- **Handle Concurrency:**

Implement optimistic concurrency control using a `RowVersion` or timestamp property to prevent data inconsistencies when multiple users modify the same data simultaneously.

- **Judicious Lazy Loading:**

While convenient, use lazy loading with caution as it can lead to performance issues (N+1 queries) if not managed properly. Explicit loading or eager loading are often preferred for performance-critical scenarios.

- **Query Filters:**

Leverage query filters for applying global predicates to entities, useful for scenarios like multi-tenancy or soft deletes.

- **Transaction Scopes:**

Use transaction scopes or explicit transactions when performing multiple related database operations to ensure atomicity and data integrity.

Module: NUnit and Moq

Here are some best practices and coding standards for using NUnit and Moq:

NUnit Best Practices:

- **Arrange-Act-Assert (AAA) Pattern:**

Structure your tests into three distinct phases:

- **Arrange:** Set up the test environment, including data and dependencies.
- **Act:** Execute the code under test.
- **Assert:** Verify the expected outcome using NUnit assertions (e.g., `Assert.AreEqual`, `Assert.IsTrue`).

- **Descriptive Test Names:**

Use clear and concise names for your test methods that indicate what functionality is being tested and what the expected outcome is (e.g., `Should_ReturnTrue_When_InputIsValid`).

- **Test Isolation:**

Ensure each test is independent and doesn't rely on the state or execution order of other tests. This makes tests more reliable and easier to debug.

- **Test Small Units:**

Focus on testing individual methods or small, isolated units of functionality.

- **Use `SetUp` and `TearDown` (or `OneTimeSetUp/OneTimeTearDown`):**

Use these attributes for common setup and cleanup operations that apply to multiple tests within a fixture.

- **Test Edge Cases:**

Include tests for boundary conditions, invalid inputs, and other edge cases to ensure robustness.

- **Automate Test Execution:**

Integrate NUnit tests into your build process and CI/CD pipeline for continuous testing.

Moq Best Practices:

- **Mock External Dependencies:**

Use Moq to create mock objects for external dependencies (databases, APIs, services) that are not part of the unit under test. This isolates the unit and allows for focused testing.

- **Mock Interfaces:**

Prefer mocking interfaces over concrete classes, as this promotes loose coupling and easier testability.

- **Keep Mocks Simple:**

Only mock the specific behavior required for the test. Avoid over-complicating mocks with unnecessary setups.

- **Verify Interactions (When Necessary):**

Use `Verify` to ensure that specific methods on a mock object were called with the expected arguments, but use it judiciously and only when the interaction itself is a critical part of the test.

- **Clear Mock Setup:**

Make your mock setups clear and readable. Avoid overly complex `Setup` chains.

General Coding Standards for Tests:

- **Readability:**

Write clear, concise, and easy-to-understand test code.

- **Consistency:**

Follow consistent naming conventions and formatting within your test suite.

- **Avoid Logic in Tests:**

Keep test methods focused on arranging, acting, and asserting. Avoid complex business logic within your tests.

- **No Magic Strings/Numbers:**

Use constants or meaningful variables instead of hardcoded values where appropriate.

- **Regular Refactoring:**

Refactor your tests as your production code evolves to maintain their effectiveness and readability.